5-Stage Pipeline Processor Simulation with Data Hazard Detection and Forwarding

Jacob Wallace
CS5365: Principles of Multiple-Processor Systems
Summer 2026
Dr. B. I. Morshed
Texas Tech University

1. Abstract

For the CS5365 Principles of Multiple-Processor Systems project, I constructed a five-stage pipeline processor simulation that runs in the Python programming language. The simulation visualizes, models, and replicates the workflow of instructions from a program through the five basic pipeline stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). The main purpose of the project is to illustrate how RAW data hazards can affect pipeline performance in processors and how stalls are inserted in the process when an instruction needs a register value that has yet to be outputted from a previous instruction. The simulation runs three separate instruction programs with different hazards: one with no hazards, one with data hazards, and one with load-use hazards. Each of these programs is simulated and tested with forwarding turned on and forwarding turned off. This ultimately will output results of six total simulations of three programs to compare the total clock cycles, number of stalls, and the cycles per instruction (CPI) for each example program. Overall, the simulation will demonstrate data hazards influence the increase of stalls and total cycles, while forwarding reduces the amount of stalls and improves pipeline performance.

2. Introduction

Pipeline processors are used to improve the throughput of instructions by allowing multiple instructions to be processed simultaneously in different stages of a processor [1]. Instead of having to wait for an instruction to finish outputting a result before starting the next instruction, a pipeline processor divides these instruction executions into separate stages. A common basic pipeline processor consists of five separate stages: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). This way, the processor allows one instruction to be fetched while another instruction is decoded, another is executed, and another accesses memory.

Although pipelining improves the performance of a processor, it can also create problems/issues when instructions depend on each other for data. A common example is a RAW data hazard, which means “Read After Write.” This occurs when an instruction needs to read a register value from a previous instruction before it has finished writing the output’s value. When this happens, a processor may need to stall the pipelining process until the required value the instruction is waiting on becomes available. The result of these stalls increase the total number of clock cycles and can reduce overall performance of a processor.

The purpose of this project is to simulate a basic five-stage pipeline processor using Python. The simulation will demonstrate and visualize how instructions move through the pipeline cycle by cycle, showing each instruction passing through at each stage. In addition, the program will also detect RAW data hazards, insert stalls in the pipeline when needed, and compare the performance of the pipeline with forwarding turned on and forwarding turned off. Forwarding is used to reduce stalls being implemented into the pipeline in order to make results available earlier for dependent instructions.

The simulation runs three different instruction programs with different hazard types: a no hazard program, a data hazard program, and a load-use hazard program. Each program will be tested in both forwarding modes, on and off. The results of the programs will be compared using total number cycles, the number of stalls, and the cycles per instruction (CPI). This way, the output will clearly illustrate how hazards can affect pipeline performance and how forwarding can improve instruction throughput of the pipeline.

3. Background

A processor pipeline is a technique used to map and improve the instruction throughput by dividing instruction execution into different stages. A non-pipelined processor for example, will need to fully complete outputting the results of an instruction before the next instruction can begin. A pipelined processor, is able to put through multiple instructions through the pipeline and can be active at the same time, with each instruction being located at a different stage throughout the pipeline. This way the processor can work on several instructions across multiple clock cycles instead of one whole instruction at a time.

This project uses a basic five stage pipeline model [1], [2]. The first stage is Instruction Fetch (IF) where the next instruction is fetched. The second stage is the Instruction Decode (ID), where the instruction is decoded and the registers needed for the instruction are checked. The third stage is Execute (EX), where the instruction performs its operation. The fourth stage is Memory Access (MEM), where the load and store instructions access memory to get appropriate values. The final fifth stage is the Write Back (WB), where the results of the instruction operation are output and written back to the destination register.

IF → ID → EX → MEM → WB

A major issue in pipelined processors is a data hazard. A data hazard occurs when an instruction depends on the outputted results on another instruction that has yet to produce its output. In this project, the simulation will focus on RAW data hazards, which stands for Read After Write. A RAW data hazard occurs when an instruction attempts to read a register value before a previous instruction has written the expected/correct output value to that register [2].

For example, if an instruction loads a value into register R1, and the next instruction after it immediately tries to read the value in R1, the second instruction likely will not be able to continue because the value for the R1 register has not yet been finalized/outputted. In this case, the pipeline will have to be stalled. A stall means that the instruction is being temporarily held in place until the needed data from the register is available and ready for use. Stalls unfortunately increase the number of clock cycles and reduce pipeline performance.

However, forwarding is a method used in order to reduce the number of stalls. Instead of having to wait until a value is fully written back to the register file, using forwarding allows the use of a later instruction to use a result earlier from another pipeline stage. This way, we can reduce waiting times for instructions that are dependent on other instructions. However, forwarding does not remove every single possible delay. Load-use hazards still can require stalls because loaded data may still not be available as early as normal ALU results [3].

Pipeline performance can be measured using the total number of cycles, the number of stalls, and cycles per instruction (CPI). A lower CPI means that the performance of the processor's pipeline is better because the processor is allowed to complete more instructions using fewer clock cycles on average. In this project, CPI will be used to compare the different program example types with forwarding turned on and off, producing six total different results.

4. Related Work

Pipeline processor simulations are commonly use to demonstrate and study how instructions move through the stages of a processor and how hazards affect the processor's performance. Hennessy and Patterson's computer architecture text explains the background behind processor performance, instruction level parallelism, and architectural design for processors [1]. Educational pipelining tools such as Ripes, can simulate processor models along with creating reports for total clock cycles, cycles per instruction (CPI), and pipeline state, similar to the output of what this project illustrates [4]. WebRISC-V is another educational tool that simulates and visualizes pipeline execution, pipeline stalls, and cycle by cycle instruction

movement throughout the pipeline [5]. More advanced tools such as gem5, provide cycle level computer architecture simulation that is much more useful and practical for bigger and more detailed systems [6]. Compared to these pre-existing tools, this project is a much smaller Python based simulator that specifically focuses on the five stage pipeline, detecting RAW data hazards, inserting stalls when needed, forwarding, and cycles per instruction (CPI) comparison.

5. Simulation Design

The simulation was designed to model a basic five stage pipeline processor. The five stages used in the simulation are the Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Each instruction moves through these five stages over multiple clock cycles. The simulation keeps track and records which instruction is in what current stage throughout each cycle and prints the information to a pipeline table.

The simulation uses three different programs to test different pipeline conditions. The first program is a no hazard example, where the instructions do not depend on one another. This program is used as a baseline example to show normal pipeline behavior/movement without stalls in a five stage processor. The second program is a data hazard example, where multiple instructions depend on one another for results from previous instructions. This program will show how RAW data hazards will cause stalls when forwarding is not available. The third program is a load-use example, where a load instruction is followed by instruction that use the loaded value from the load instruction. This program will show how load-use dependencies can affect a pipeline processor's performance.

Each of these three programs will be executed through the 5 stage pipeline processor twice. The first run will simulate all three programs with forwarding turned off, and then the second round with all three programs being put through the pipeline processor with forwarding turned on. This will output six total simulations with their corresponding results. Running both modes will allow the results to show how forwarding impacts changes to the total number of cycles, stalls and cycles per instruction (CPI).

During each clock cycle, the simulation will check on whether instructions are able to progress forward a stage through the pipeline. Instructions in the EX stage will move forward to MEM, and instructions in the MEM stage will move forward to the WB stage. The ID stage is where the simulator checks for RAW data hazards, so if an instruction needs a source register that is not ready at that moment, the instruction remains in the ID stage and a stall is recorded. Otherwise, if there is no RAW data hazard, the instruction moves on to the next EX stage.

The simulator will keep track of the destination register and when they are ready to be available. When forwarding is turned off, the instructions that depend on another will have to

wait longer as the results are not available until later in the pipeline. When forwarding is turned on, the results will be made available earlier, which will ultimately allow dependent instructions to continue their operations faster/sooner. This will ultimately reduce the number of stalls in the data hazard program and the load-use hazard programs.

At the end of each simulation, the program will print the instruction list, the cycle by cycle pipeline table with each stage, the stall details, and a summary of the results from the program going through the five stage processor's pipeline. The summary will include the number of instructions, total cycle count, total stalls, and the cycles per instruction (CPI). After all six of the simulations are fully complete, the simulation will ultimately print a comparison summary showing the differences of all three programs with and without forwarding.

6. Implementation

The simulation is implemented in the Python programming language. The program is organized into the main parts: defining the test programs, setting up the pipeline stages, running the cycle by cycle simulation, checking for hazards, handling stalls, applying forwarding logic, and printing the final results.

6.1 Program Inputs

The program begins by initializing three instruction programs, each program demonstrating different data hazards. Each instruction is stored into a python dictionary that contains the operation type, the destination register, source registers, offset, and base register. The operations the program supports in the simulations are LW, SW, ADD, SUB, and AND.

The first example instruction set is the no hazard program. This program is the baseline because the instructions do not depend on one another. The second set of instructions is the data hazard program. In this example, later instructions will use registers that were produced by earlier instructions. The third set of instructions is a load use hazard program. This program includes a load instruction followed by an immediate dependency on the loaded value.

6.2 Forwarding Runs

Each program that is tested will be executed twice. The first run is with forwarding turned off, and the second run is with forwarding turned on. This will allow the simulator to compare how the same instruction program examples perform in both conditions for forwarding being on and off. For each run, the program will store the pipeline table, total cycles, stall list, stall messages, and forwarding mode.

6.3 Pipeline Stage Table

The pipeline of a five stage processor is represented by the stage: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). Each stage is stored in a dictionary called stage, where every stage contains a list that tracks which instruction is in that stage at each clock cycle. If a stage does not contain an instruction during a cycle, the stage is stored as empty.

This design allows the program to print cycle by cycle representation of each stage in a table at the end of each simulation. Each row of the table represents one clock cycle and each column represents a pipeline stage.

6.4 Main Clock Cycle Loop

The main simulation of the program is controlled by a while loop. Each loop iteration of the while loop represents one clock cycle. The while loop will continue to loop until all the instructions have been fetched and have successfully gone through the pipeline leaving each stage empty.

During each cycle in the simulator, instructions move forward through the pipeline's stages when possible. An instruction in the MEM stage will move to the WB stage, and the instruction in the EX stage will move to the MEM stage. The simulator will then check the instruction in the ID stage to decide whether the instruction is able to move into the EX stage or stall.

6.5 Hazard Detection

Hazard detections happen in the ID stage. The program will first determine which source registers are needed by the current instruction. A load instruction is assigned to reading the base register. A store instruction is assigned to reading the value being stored and the base register. Otherwise, arithmetic and logic instructions are assigned/required to read two source registers.

The program uses a dictionary named "ready" to track when each destination register will become available. If a source register is found ready and is not ready to be available fast/soon enough, the simulator will detect a RAW data hazard. This means that the current instruction needs a value that a past instruction has not yet finished outputting.

6.6 Stall Handling

When a RAW data hazard is detected, the instruction will stay in the ID stage for another cycle. This will create a stall, which the simulator records and stores in a list and creates a

corresponding message showing which instruction stalled and which register had not yet had the result.

If the ID stage is blocked by a stall, the IF stage will be held back as well. This demonstrates how one stalled instruction can delay and prolong all the other instructions that follow in the pipeline.

6.7 Forwarding Logic

Forwarding changes when the destination registers become available. When forwarding is turned off, the results are treated as they will be ready/available later, so dependent instructions will have to wait longer. This will cause more stalls to be created.

When forwarding is turned on, some of the results are made available/ready earlier. ALU instructions such as ADD, SUB, and AND, can provide results sooner. Load instructions however, will still take longer because the loaded data is not yet available as a normal ALU result is. This is why forwarding reduces stalls but unfortunately will not completely remove every delay.

6.8 Output Generation

After each simulation finishes inputting the instruction set examples, the program will print the instruction sequence, the pipeline table with stages, the stall details, and a summary. The summary includes the total number of instructions, total cycles count, total stalls, and the cycles per instruction (CPI).

After all three of the programs have successfully been completed with forwarding off and on, the program will ultimately print a final comparison summary. This table displays the cycles and stall for each program with both forwarding on and forwarding off. This makes the results easier to compare the performance differences caused by forwarding.

7. Results

The simulation was tested using three different instruction programs. Each program was executed twice: once with forwarding turned off and once with forwarding turned on. The results were compared using total clock cycles, total stalls, and cycles per instruction (CPI).

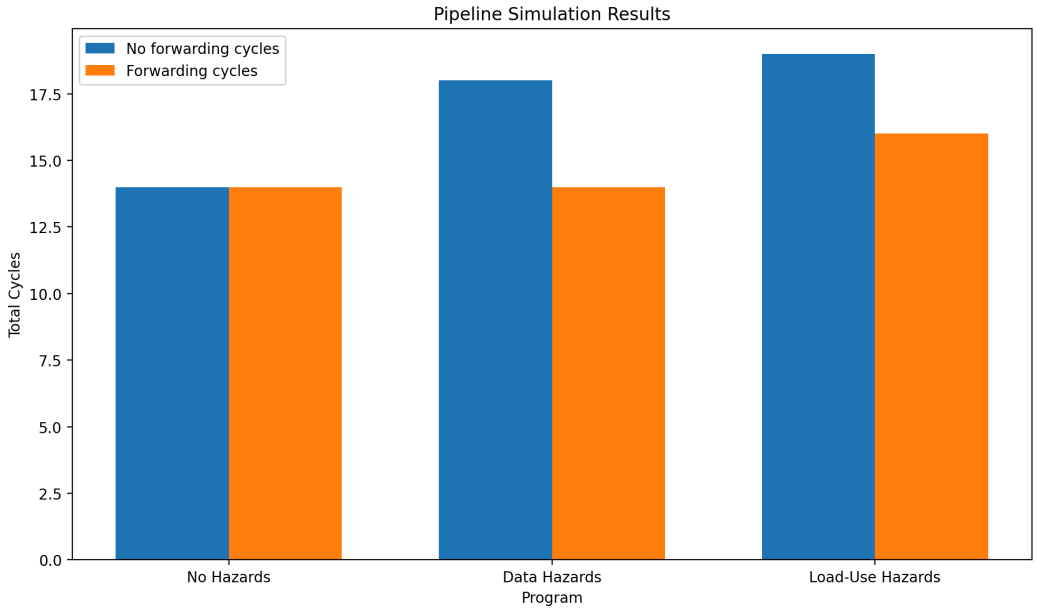


Figure 1: Total cycle comparison with forwarding turned off and forwarding turned on.

Program	No Forwarding Cycles	No Forwarding Stalls	No Forwarding CPI	Forwarding Cycles	Forwarding Stalls	Forwarding CPI
No Hazards	14	0	2.80	14	0	2.80
Data Hazards	18	7	3.60	14	1	2.80
Load-Use Hazards	19	7	3.17	16	1	2.67

Figure 2. Simulation results comparing cycles, stalls, and CPI with forwarding off and forwarding on.

The no hazards program gives the same result with forwarding on and off. This happens because the instructions have no dependency on one another for register results from previous instructions. Since there were no existing RAW data hazards, no stalls are needed for this program.

The data hazards program showed improvement with forwarding turned on versus when forwarding was off. Without forwarding the program required 18 cycles and 7 stalls, compared to the 14 cycles and 1 stall resulting from forwarding turned on. This shows that forwarding allows instructions that are dependent on one another to receive the needed register values from other

earlier instructions faster and more effectively, which reduces the amount of stalls needed and lowers the total cycle count.

The load use hazard program had also shown improvement with forwarding turned on. Without forwarding, the program needed 19 cycles and 7 stalls, compared to the 16 cycles and 1 stall with forwarding turned on. These results again show to help that forwarding reduces the number of stalls, although load use hazards can still create a delay as loaded values are available later than typical ALU results.

Overall, the results from the figures depict that RAW data hazards increase the number of stall and total clock cycles during the pipeline process. Forwarding improves the pipeline by reducing stalls, decreasing total cycles, and improving overall clocks per instruction (CPI).

8. Analysis

The results of the simulation show that forwarding has a big impact when instructions are dependent on the results of previous instructions. The no hazard program with forwarding had no change on the total cycles, stalls, or the cycles per instruction (CPI). Because the instructions were independent of each other, no instruction had to wait for previous instructions to finish producing their register value. Since no RAW data hazards were present, the simulation did not have to insert any stalls into the pipeline.

The data hazard program showed a clear difference of results compared to when forwarding was off and when forwarding was on. Without forwarding, the program required 18 total cycles and 7 stalls. With forwarding turned on, the program needed a total of 14 cycles and only one single stall. This happened because forwarding enabled dependent instructions get results from needed register values earlier instead of having to wait until the write back (WB) stage. As a result, less instructions needed to remain stalled in the instruction decode (ID) stage, which was able to reduce the total number of stalls and improved the overall cycles per instruction (CPI) from 3.60 to 2.80.

The load-use hazard program had also shown improvements when forwarding was turned on. Without forwarding, the program needed 19 total cycles and 7 stalls. However, with forwarding turned on, the program instead needed 16 cycles total and one single stall. This result shows that forwarding still is useful when load use hazards exist, but it does not completely remove every delay that occurs. A load instruction can take longer to make its results available due to the fact that the value is coming from memory, so a dependent instruction still may need to wait.

Overall, the simulation showed that RAW data hazards reduce the pipeline performance of the processor simulation by increasing the stalls and the total clock cycles. With forwarding, the performance improves by making the results of the register values available faster/sooner for dependent instructions. This ultimately lowers the amount of stalls, reduces the total cycle count, and improves the cycles per instruction (CPI). The results show that the benefit of forwarding depends on the type of program being put through the processor pipeline. If there are no dependencies between the instructions within the program, forwarding has no effect on whether the results improve. If data dependencies do exist in a program, forwarding can significantly improve the throughput of instructions through the processor's pipeline.

9. Conclusion

This project implements a five stage pipeline processor simulation using Python. The simulation models how instructions in a program move through the processor's pipeline. Visualizing how each instruction progresses through the five stages Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB). The simulation also is capable of detecting RAW data hazards, stalls that are inserted when needed, and compared pipeline performances with forwarding turned off and forwarding turned on for each program simulated.

The results of the simulation shows that data hazards can reduce pipeline performances by increasing the number of stalls and the total clock cycles. For the no hazard program, forwarding had made no change on the results because the instructions were independent of each other and did not have to rely on previous instructions for register values. In the data hazard program and load use program, forwarding had shown to reduce the number of stalls and had lowered the total cycle count compared to when forwarding was off. This also improved the cycles per instruction (CPI), showing that the pipeline was able to efficiently complete more instructions faster when forwarding was available.

Overall, the simulation of the five stage pipeline processors demonstrates how pipeline hazards affect the processor's performance and how forwarding can reduce the performance loss that are caused by dependent instructions in a program. The project helps illustrate the importance of hazard detection, handling stalls, and forwarding in pipelined processor designs.

10. References

[1] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Morgan Kaufmann/Elsevier, 2017.
[2] CS61C Course Notes, "Intro to Hazards," University of California, Berkeley.
[3] CS61C Course Notes, "Data Hazards," University of California, Berkeley.

[4] M. B. Petersen, “Ripes Documentation: Command-line Interface.”
[5] R. Giorgi and G. Mariotti, “WebRISC-V: A 64-bit RISC-V Pipeline Simulator for Computer Architecture Classes,” 2025.
[6] J. Lowe-Power et al., “The gem5 Simulator: Version 20.0+,” 2020.

11. Appendix

Appendix A: Python Source Code

The following source code provided below was used to implement the five stage pipeline processor simulation. The python program initializes and defines three test programs each with different data hazards. The program runs each test program with forwarding turned off and forwarding turned on, tracks and displays the pipeline stages for each clock cycle, detects RAW data hazards, stalls inserted when needed, and prints the output of the final cycles, stalls, and cycles per instruction (CPI) results.

```
#Jacob Wallace - R11822561
#============================================================================
==========================
#Summer 2026 TTU Principles of Multiple-Processor Systems (CS-5365-202 & D01)
1st Summer Session Project
#5 stage pipeline processor simulation with data hazard detection and
forwarding
#============================================================================
==========================

#============================================================================
==========================
#                                          Overview:
#============================================================================
==========================
#1.Programs - creates three instruction sets for different pipeline cases
#2.Forwarding Runs - runs each test with forwarding off and on
#3.Setup Pipeline - stores IF, ID, EX, MEM, and WB for each cycle
#4.Cycle Simulation - moves instructions through the pipeline stages
#5.Hazards - checks source registers for RAW data hazards
#6.Stalls - holds instructions when a needed register is not ready
#7.Forwarding - changes timing when destination registers become ready
#8.Output - prints instructions, pipeline table, stalls, and CPI of all
programs
#============================================================================
==========================


programs = [

    #1 - 1st test: instructions that do not depend on each other
```

```
    ("No Hazards", [
        {"op": "LW",  "dest": "R1", "offset": 0, "base": "R2"},
        {"op": "ADD", "dest": "R3", "src1": "R4", "src2": "R5"},
        {"op": "SUB", "dest": "R6", "src1": "R7", "src2": "R8"},
        {"op": "AND", "dest": "R9", "src1": "R10", "src2": "R11"},
        {"op": "SW",  "src1": "R12", "offset": 4, "base": "R2"}
    ]),
    #1.1 - 2nd test: instructions need earlier results
    ("Data Hazards", [
        {"op": "LW",  "dest": "R1", "offset": 0, "base": "R2"},
        {"op": "ADD", "dest": "R3", "src1": "R1", "src2": "R4"},
        {"op": "SUB", "dest": "R5", "src1": "R3", "src2": "R6"},
        {"op": "AND", "dest": "R7", "src1": "R5", "src2": "R8"},
        {"op": "SW",  "src1": "R7", "offset": 4, "base": "R2"}
    ]),
    #1.2 - 3rd test: load-use hazard example w/ additional instruction to show
forwarding effect
    ("Load Use Hazards", [
        {"op": "LW",  "dest": "R1", "offset": 0, "base": "R2"},
        {"op": "ADD", "dest": "R3", "src1": "R1", "src2": "R4"},
        {"op": "ADD", "dest": "R5", "src1": "R3", "src2": "R6"},
        {"op": "SUB", "dest": "R7", "src1": "R5", "src2": "R8"},
        {"op": "SW",  "src1": "R7", "offset": 8, "base": "R2"},
        {"op": "ADD", "dest": "R9", "src1": "R1", "src2": "R10"}

    ]),
]


results = []

#2 - each program runs with forwarding off and on
for item in programs:
    title=item[0]
    program=item[1]
    runs = [] #hold results for forwarding

     #setup pipeline stages
    for forwarding in [False, True]:
        count = len(program)

        stage = {}
        stages =["IF", "ID", "EX", "MEM", "WB"]

        #3 - five stages are saved so every cycle can be tracked
        for i in stages:
            stage[i] = [None] * (count*6) #multiply by 6 to have enough cycles
for the program to finish processing

        # variables to track pipeline state
        ready = {} #hazard detection: tracks when registers are ready to be
read
        stalls = []
        messages = []
        cycleCount = 0
```

```
        next=0

        #4 - main clock cycle loop for the simulation
        while True:

            active=False

            #4.1 - checks if the pipeline still has an instruction inside it
            for i in stage:
                if (stage[i][cycleCount] is not None):
                    active=True
                    break

            if (next >=count and not active):
                break

            cycleCount = cycleCount+1

            #4.2 - MEM and EX move foward to the writeback stage
            if (stage["MEM"][cycleCount-1] is not None):
                stage["WB"][cycleCount] = stage["MEM"][cycleCount-1]
            if (stage["EX"][cycleCount-1] is not None):
                stage["MEM"][cycleCount] = stage["EX"][cycleCount-1]




            #5 - ID stage checks source registers for hazards
            if (stage["ID"][cycleCount-1] is not None):
                index = stage["ID"][cycleCount-1] #index of the instruction in
the program list
                instr = program[index] #instruction data for the current
instruction in ID stage

                #5.1 - different instruction types read different registers
                if (instr["op"]=="LW"):
                    sources =[instr["base"]]
                elif (instr["op"]=="SW"):
                    sources = [instr["src1"], instr["base"]]
                elif (instr["op"]=="ADD" or instr["op"]=="SUB" or
instr["op"]=="AND"):
                    sources = [instr["src1"], instr["src2"]]
                else:
                    sources=[]

                #print(f"Cycle {cycleCount}: I{index + 1} sources: {sources}")

                stall = False
                stallreg = ""

                #5.2 - stall is needed if a source register is not ready
                for reg in sources: #pull each source register and check if it
is ready
                    if (reg in ready and ready[reg] >= cycleCount + 1):
                        stall = True
```

```
                        stallreg = reg
                        break
                if (stall):
                    #6 - instruction stays in ID and the stall is saved
                    stage["ID"][cycleCount] = index
                    #print(stage["ID"][cycleCount])
                    stalls.append((cycleCount, index))
                    messages.append(f"Cycle {cycleCount}: I{index + 1} stalled!
{stallreg} not ready!")
                else: #no stall is needed and the instruction can move forward
                    #6.1 - no hazard: instruction can move into EX
                    stage["EX"][cycleCount] = index

                     #7 - store instructions do not write to a destination
register
                    if (instr["op"] == "SW"): #store does not have a
destination register
                        dest = None
                    else:
                        dest = instr.get("dest")


                    if (dest is not None): #if instruction has destination
register, update the ready cycle for that register
                        #7.1 - forwarding to make the results available faster
by changing the ready cycle for the destination register
                        if (forwarding):
                            if (instr["op"] == "LW"): #load instruction results
are ready later than ALU instructions
                                ready[dest] = cycleCount+3 #INC by 3 cycles for
faster results
                            else: # ALU instruction results are ready sooner
                                ready[dest] = cycleCount+1 #default

                        else: #7.2 - without forwarding, results are ready
later
                            ready[dest] = cycleCount+3 #INC by 3 cycles for
faster results

        #4.3 - IF stage move forward only if ID is open
            if (stage["IF"][cycleCount-1] is not None and
stage["ID"][cycleCount] is None):
                index = stage["IF"][cycleCount-1]
                stage["ID"][cycleCount] = index
            elif (stage["IF"][cycleCount-1] is not None):
                stage["IF"][cycleCount] = stage["IF"][cycleCount-1]

            #4.4 - new instruction is fetched when IF is open
            if (next < count and stage["IF"][cycleCount] is None and
stage["IF"][cycleCount-1] is None):
                stage["IF"][cycleCount] =next
                next = next+1

        #8 - store the results for the current run of the program
        result = {}
```

```
        result["stage"] = stage
        result["cycleCount"] = cycleCount
        result["stalls"] = stalls
        result["stallMessages"] = messages
        result["program"] = program
        result["forwarding"] = forwarding
        runs.append(result) #append run to list of recorded runs for the
program

        print("\n==================================================")
        print(f"PROGRAM: {title}")
        print(f"Forwarding: {'ON' if forwarding else 'OFF'}")
        print("==================================================")
        print("\nPROGRAM:")

        index = 0
        for instr in program:  #8.1 - instructions print before the pipeline
table
            op = instr["op"]

            #8.2 - stored instruction data changes into assembly text
            if (op == "LW"):
                line = f"LW {instr['dest']},
{instr['offset']}({instr['base']})"
            elif (op == "SW"):
                line = f"SW {instr['src1']},
{instr['offset']}({instr['base']})"
            elif (op == "ADD" or op == "SUB" or op == "AND"):
                line = f"{op} {instr['dest']}, {instr['src1']},
{instr['src2']}"
            else: #error
                line = "[ERROR] Unknown instruction!"

            print(f"I{index + 1}: {line}" )
            index=index+1


        print("\nTABLE:")
        print(f"Cycle | IF | ID | EX | MEM | WB |")

        #8.3 - table shows each clock cycle and instruction position
        for i in range(1, cycleCount + 1):
            row = f"{i:<5} |"
            for j in ["IF", "ID", "EX", "MEM", "WB"]:
                spot = stage[j][i] #j is the stage, i is the cycle for that
stage
                position = f"I{spot + 1}" if spot is not None else "-"
                if (j != "MEM"):
                    row += f" {position:<2} |"
                else:
                    row += f" {position:<3} |"
            print(row)

         #8.4 - stall details explain why hazards happened
        print("\nSTALL DETAILS:")
```

```
        if (len(messages)==0):
            print("None")
        else:
            for message in messages:
                print(message)

        #8.5 - print summary: cycleCount, stalls, and CPI
        print("\nSUMMARY:")
        print(f"Instructions: {len(program)}")
        print(f"Total Cycles: {cycleCount}")
        print(f"Stalls: {len(stalls)}")
        print(f"CPI: {cycleCount / len(program):.2f}\n")

    resultOff = runs[0]
    resultOn = runs[1]
    results.append((title, resultOff, resultOn))


#8.6 - print the results and show how forwarding effected the cycles and stalls
print("\nCOMPARISON SUMMARY:")
print(f"{'Program':<20} | {'No Fwd Cycles':<13} | {'No Fwd Stalls':<13} | {'Fwd
Cycles':<10} | {'Fwd Stalls':<10}")
for item in results:

    title = item[0] #print title
    resultOff = item[1] #print results (forwarding off)
    resultOn = item[2] #print results (forwarding on)

    print(f"{title:<20} | {resultOff['cycleCount']:<13} |
{len(resultOff['stalls']):<13} | {resultOn['cycleCount']:<10} |
{len(resultOn['stalls']):<10}")
```

Appendix B: Full Program Output

The following output was generated by running the Python simulation in Visual Studio Code. The output includes each test program, the cycle-by-cycle pipeline table, stall details, CPI values, and the final comparison summary.

```
================================================
PROGRAM: No Hazards
Forwarding: OFF
================================================

PROGRAM:
I1: LW R1, 0(R2)
I2: ADD R3, R4, R5
I3: SUB R6, R7, R8
I4: AND R9, R10, R11
I5: SW R12, 4(R2)

TABLE:
Cycle | IF | ID | EX | MEM | WB |
1     | I1 | -  | -  | -   | -  |
```

```
2     | -  | I1 | -  | -   | -  |
3     | I2 | -  | I1 | -   | -  |
4     | -  | I2 | -  | I1  | -  |
5     | I3 | -  | I2 | -   | I1 |
6     | -  | I3 | -  | I2  | -  |
7     | I4 | -  | I3 | -   | I2 |
8     | -  | I4 | -  | I3  | -  |
9     | I5 | -  | I4 | -   | I3 |
10    | -  | I5 | -  | I4  | -  |
11    | -  | -  | I5 | -   | I4 |
12    | -  | -  | -  | I5  | -  |
13    | -  | -  | -  | -   | I5 |
14    | -  | -  | -  | -   | -  |

STALL DETAILS:
None

SUMMARY:
Instructions: 5
Total Cycles: 14
Stalls: 0
CPI: 2.80


================================================
PROGRAM: No Hazards
Forwarding: ON
================================================

PROGRAM:
I1: LW R1, 0(R2)
I2: ADD R3, R4, R5
I3: SUB R6, R7, R8
I4: AND R9, R10, R11
I5: SW R12, 4(R2)

TABLE:
Cycle | IF | ID | EX | MEM | WB |
1     | I1 | -  | -  | -   | -  |
2     | -  | I1 | -  | -   | -  |
3     | I2 | -  | I1 | -   | -  |
4     | -  | I2 | -  | I1  | -  |
5     | I3 | -  | I2 | -   | I1 |
6     | -  | I3 | -  | I2  | -  |
7     | I4 | -  | I3 | -   | I2 |
8     | -  | I4 | -  | I3  | -  |
9     | I5 | -  | I4 | -   | I3 |
10    | -  | I5 | -  | I4  | -  |
11    | -  | -  | I5 | -   | I4 |
12    | -  | -  | -  | I5  | -  |
13    | -  | -  | -  | -   | I5 |
14    | -  | -  | -  | -   | -  |

STALL DETAILS:
None
```

```
SUMMARY:
Instructions: 5
Total Cycles: 14
Stalls: 0
CPI: 2.80


================================================
PROGRAM: Data Hazards
Forwarding: OFF
================================================

PROGRAM:
I1: LW R1, 0(R2)
I2: ADD R3, R1, R4
I3: SUB R5, R3, R6
I4: AND R7, R5, R8
I5: SW R7, 4(R2)

TABLE:
Cycle | IF | ID | EX | MEM | WB |
1     | I1 | -  | -  | -   | -  |
2     | -  | I1 | -  | -   | -  |
3     | I2 | -  | I1 | -   | -  |
4     | -  | I2 | -  | I1  | -  |
5     | I3 | I2 | -  | -   | I1 |
6     | -  | I3 | I2 | -   | -  |
7     | I4 | I3 | -  | I2  | -  |
8     | I4 | I3 | -  | -   | I2 |
9     | -  | I4 | I3 | -   | -  |
10    | I5 | I4 | -  | I3  | -  |
11    | I5 | I4 | -  | -   | I3 |
12    | -  | I5 | I4 | -   | -  |
13    | -  | I5 | -  | I4  | -  |
14    | -  | I5 | -  | -   | I4 |
15    | -  | -  | I5 | -   | -  |
16    | -  | -  | -  | I5  | -  |
17    | -  | -  | -  | -   | I5 |
18    | -  | -  | -  | -   | -  |

STALL DETAILS:
Cycle 5: I2 stalled! R1 not ready!
Cycle 7: I3 stalled! R3 not ready!
Cycle 8: I3 stalled! R3 not ready!
Cycle 10: I4 stalled! R5 not ready!
Cycle 11: I4 stalled! R5 not ready!
Cycle 13: I5 stalled! R7 not ready!
Cycle 14: I5 stalled! R7 not ready!

SUMMARY:
Instructions: 5
Total Cycles: 18
Stalls: 7
CPI: 3.60
```

```
================================================
PROGRAM: Data Hazards
Forwarding: ON
================================================

PROGRAM:
I1: LW R1, 0(R2)
I2: ADD R3, R1, R4
I3: SUB R5, R3, R6
I4: AND R7, R5, R8
I5: SW R7, 4(R2)

TABLE:
Cycle | IF | ID | EX | MEM | WB |
1     | I1 | -  | -  | -   | -  |
2     | -  | I1 | -  | -   | -  |
3     | I2 | -  | I1 | -   | -  |
4     | -  | I2 | -  | I1  | -  |
5     | I3 | I2 | -  | -   | I1 |
6     | -  | I3 | I2 | -   | -  |
7     | I4 | -  | I3 | I2  | -  |
8     | -  | I4 | -  | I3  | I2 |
9     | I5 | -  | I4 | -   | I3 |
10    | -  | I5 | -  | I4  | -  |
11    | -  | -  | I5 | -   | I4 |
12    | -  | -  | -  | I5  | -  |
13    | -  | -  | -  | -   | I5 |
14    | -  | -  | -  | -   | -  |

STALL DETAILS:
Cycle 5: I2 stalled! R1 not ready!

SUMMARY:
Instructions: 5
Total Cycles: 14
Stalls: 1
CPI: 2.80


================================================
PROGRAM: Load Use Hazards
Forwarding: OFF
================================================

PROGRAM:
I1: LW R1, 0(R2)
I2: ADD R3, R1, R4
I3: ADD R5, R3, R6
I4: SUB R7, R5, R8
I5: SW R7, 8(R2)
I6: ADD R9, R1, R10

TABLE:
```

```
Cycle | IF | ID | EX | MEM | WB |
1     | I1 | -  | -  | -   | -  |
2     | -  | I1 | -  | -   | -  |
3     | I2 | -  | I1 | -   | -  |
4     | -  | I2 | -  | I1  | -  |
5     | I3 | I2 | -  | -   | I1 |
6     | -  | I3 | I2 | -   | -  |
7     | I4 | I3 | -  | I2  | -  |
8     | I4 | I3 | -  | -   | I2 |
9     | -  | I4 | I3 | -   | -  |
10    | I5 | I4 | -  | I3  | -  |
11    | I5 | I4 | -  | -   | I3 |
12    | -  | I5 | I4 | -   | -  |
13    | I6 | I5 | -  | I4  | -  |
14    | I6 | I5 | -  | -   | I4 |
15    | -  | I6 | I5 | -   | -  |
16    | -  | -  | I6 | I5  | -  |
17    | -  | -  | -  | I6  | I5 |
18    | -  | -  | -  | -   | I6 |
19    | -  | -  | -  | -   | -  |

STALL DETAILS:
Cycle 5: I2 stalled! R1 not ready!
Cycle 7: I3 stalled! R3 not ready!
Cycle 8: I3 stalled! R3 not ready!
Cycle 10: I4 stalled! R5 not ready!
Cycle 11: I4 stalled! R5 not ready!
Cycle 13: I5 stalled! R7 not ready!
Cycle 14: I5 stalled! R7 not ready!

SUMMARY:
Instructions: 6
Total Cycles: 19
Stalls: 7
CPI: 3.17


================================================
PROGRAM: Load Use Hazards
Forwarding: ON
================================================

PROGRAM:
I1: LW R1, 0(R2)
I2: ADD R3, R1, R4
I3: ADD R5, R3, R6
I4: SUB R7, R5, R8
I5: SW R7, 8(R2)
I6: ADD R9, R1, R10

TABLE:
Cycle | IF | ID | EX | MEM | WB |
1     | I1 | -  | -  | -   | -  |
2     | -  | I1 | -  | -   | -  |
3     | I2 | -  | I1 | -   | -  |
```

```
4     | -  | I2 | -  | I1  | -  |
5     | I3 | I2 | -  | -   | I1 |
6     | -  | I3 | I2 | -   | -  |
7     | I4 | -  | I3 | I2  | -  |
8     | -  | I4 | -  | I3  | I2 |
9     | I5 | -  | I4 | -   | I3 |
10    | -  | I5 | -  | I4  | -  |
11    | I6 | -  | I5 | -   | I4 |
12    | -  | I6 | -  | I5  | -  |
13    | -  | -  | I6 | -   | I5 |
14    | -  | -  | -  | I6  | -  |
15    | -  | -  | -  | -   | I6 |
16    | -  | -  | -  | -   | -  |

STALL DETAILS:
Cycle 5: I2 stalled! R1 not ready!

SUMMARY:
Instructions: 6
Total Cycles: 16
Stalls: 1
CPI: 2.67


COMPARISON SUMMARY:
Program              | No Fwd Cycles | No Fwd Stalls | Fwd Cycles | Fwd Stalls
No Hazards           | 14            | 0             | 14         | 0
Data Hazards         | 18            | 7             | 14         | 1
Load Use Hazards     | 19            | 7             | 16         | 1
```